Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 2

Algoritmi recursivi. Liste liniare simplu inlantuite.

1. **Recursivitate. Algoritmi autoapelabili.**
2. **Liste liniare simplu inlantuite**
   - 2.1. **Introducere**
   - 2.2. **Liste liniare simplu inlantuite alocate static**
   - 2.3. **Liste liniare simplu inlantuite alocate dinamic**
   - 2.4. **Operatii in liste liniare simplu inlantuite**

1. Recursivitate. Algoritmi autoapelabili

Introducere

Recursivitatea este o notiune fundamentala in matematica. Caracteristica esentiala a unei definitii recursive consta in referirea in enunt la obiectul definit.

Algoritmii recursivi sunt ***algoritmi care se autoapeleaza***.

Executia unei functii care implementeaza un algoritm recursiv se face astfel:

- se creeaza in stiva de recursie o "inregistrare de activare" in care sunt memorati:
  - parametrii de apel;
  - adresa instructiunii de retur (cu care va continua programul dupa terminarea executiei functiei);
- se rezerva spatiu pentru variabile locale.
- se executa instructiunile functiei care folosesc pentru parametri si variabile locale parametrii memorati in "inregistrarea de activare";
- se scoate din stiva "inregistrarea de activare" (decrementarea varfului stivei), stiva fiind ordonata; se continua cu instructiunea data de adresa de retur memorata in "inregistrarea de activare".

Asadar, *variabilele globale (statice)* sunt memorate intr-o zona de memorie fixa, mai exact in segmentele de date. *Variabilele automate (locale)* se memoreaza in stiva, iar *variabilele dinamice* in "heap"-uri (cu malloc in C, si cu new in C++).

Consumul de memorie al algoritmului recursiv este proportional cu numarul de apeluri recursive ce se fac. Variabilele recursive consuma mai multa memorie decat cele iterative. La prelucrarea unei liste, daca primul element nu este vid, se prelucreaza acesta, urmand apoi ca restul listei sa fie considerat ca o noua lista mai mica, etc.

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 2

Un exemplu simplu de functie recursiva este factorial(), care calculeaza factorialul unui numar intreg. In contiunuare sunt prezentate ambele variante, recursiva si iterativa, ale acestei functii :

```
/* recursiv */
int r_factorial(int n) {
     int answer;
     if(n==1)
          return(1);
     answer = factorial(n-1)*n; /* apel recursiv */
     return(answer);
}

/* iterativ */
int i_factorial(int n) {
     int t, answer;
     answer = 1;
     for(t=1; t<=n; t++)
          answer=answer*(t);
     return(answer);
}
```

Cand o functie se autoapeleaza, se aloca spatiu pe stiva pentru un nou set de parametri si variabile locale, iar codul functiei este executat de la inceput cu aceste noi variabile. Un apel recursiv nu determina crearea unei copii a functiei. Se creeaza numai copii ale valorilor asupra carora se opereaza. La returul din fiecare apel recursiv vechii parametri si variabile locale sunt inlaturati din stiva iar executia se reia din punctul din care s-a facut apelul recursiv.

Majoritatea rutinelor recursive nu reduc considerabil dimensiunea codului si nici nu imbunatatesc utilizarea memoriei. De asemenea, versiunile recursive ale unor rutine se executa mai incet decat variantele iterative datorita intarzierii introduse de apelurile repetate ale functiei. De fapt, multe apeluri recursive ale unei functii ar putea cauza umplerea stivei (stack overrun), deoarece stocarea parametrilor si a variabilelor locale se face pe stiva, iar fiecare nou apel creeaza noi copii ale acestor variabile. Totusi, aceste probleme ar putea aparea doar daca o functie recursiva scapa de sub control.

Principalul **avantaj** al functiilor recursive este acela ca pot fi folosite pentru a crea versiuni mai clare si mai simple ale mai multor algoritmi. De exemplu, algoritmul QuickSort este dificil de implementat in varianta iterativa. De asemenea, unele probleme, in special cele legate de inteligenta artificiala, prezinta doar solutii recursive. In cele din urma, unii oameni se pare ca gandesc mai bine recursiv decat iterativ.

Atentie! La scrierea unei functii recursive, trebuie inclusa o instructiune conditionala, precum un **if**, pentru a forta returul functiei fara a se mai executa un apel recursiv. Fara aceasta instructiune, odata apelata functia, executia ei nu se va termina niciodata. Omiterea instructiunii conditionale reprezinta o eroare comuna in scrierea functiilor recursive.

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 2

Aplicatii

Problema turnurilor din Hanoi

Se dau ***n*** discuri: a_1, a_2, ... , a_n de dimensiuni diferite, cu $d_1 < d_2 < ... < d_n$, d_i - fiind diametrul discului a_i. Discurile respective sunt stivuite pe o tija.
Se cere sa se deplaseze aceasta stiva pe o alta tija, folosind ca manevra o tija auxiliara, respectandu-se conditia: Un disc nu poate fi plasat decat peste un disc cu diametrul mai mare decat al acestuia.
Problema P(n) a deplasarii a ***n*** discuri, se rezolva prin deplasari succesive ale discurilor de pe o tija pe alta. Deplasarea de pe o tija pe alta este echivalenta cu deplasarea a ***n-1*** discuri de pe tija intiala (t_i) pe tija de manevra (t_m), apoi plasarea celui mai mare disc pe tija finala (t_f), pentru ca la sfarsit sa se aduca de pe tija de manevra pe tija finala cele ***n-1*** discuri.

PseudoCod:

```
Hanoi(n, ti, tf, tm)
{
     if(n=1) then muta (ti, tf) //deplaseaza discul superior
                                //de pe ti pe tf
     else  Hanoi(n-1, ti, tm, tf)
             muta(ti, tf)
             Hanoi(n-1, tm, tf, ti)
}
```

Pentru o problema P(1), timpul T(1) = 1 , pentru o mutare.

Pentru P(n) , timpul:

$$T(n)=2*T(n-1)+1 \qquad (1)$$

Dorim sa aflam ordinul de complexitate a lui T(n). Asociem relatiei (1) ecuatia caracteristica:

$$x=2x+1 \qquad (2)$$

Rezulta $x_0=-1$ si $T(n)- x_0=2*T(n-1)- x_0$

Raca se noteaza $f(n)=T(n)- x_0$ se obtine

$$f(n)=2*f(n-1) \qquad (3)$$

Aplicand (2) rezulta $f(n)=2*f(n-1)=2*2*f(n-2)=...=2^{n-1} * f(1)$.

Inlocuind f(n) cu T(n)+1 si f(1) cu 2 se obtine $T(n)=2^n -1$.

Deci ordinul de complexitate este ***$O(2^n)$***, adica o complexitate exponentiala.

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 2

Problema Jeep-urilor

Un jeep are un rezervor care poate contine ***r*** litri de benzina, consuma ***c*** litri de benzina la 100 km. Initial jeep-ul este parcat intr-o oaza din Sahara si are rezervorul gol. In oaza exista ***n*** canistre, fiecare continind ***r*** litri de benzina. Presupunem ca jeep-ul poate transporta la un moment dat o singura canistra plus benzina din rezervor. Rezervorul poate fi umplut numai atunci cand este golit complet.

Sa se determine cea mai mare distanta fata de oaza (pozitia initiala) care poate fi parcursa de jeep utilizand toate cele ***n*** canistre.

APLICAȚII

1. Sa se construiasca un modul C (C++) pentru problema Turnurilor din Hanoi.

2. Sa se scrie un algoritm recursiv pentru rezolvarea problemei jeep-urilor.

TEMA

Problema labirintului

Se da o matrice cu elemente 0 si 1 reprezentind "harta" unui labirint (0 - spatiu liber; 1 -zid). Se cere sa se determine un drum intre o pozitie initiala si o pozitie finala date. Un drum este format din pozitii libere invecinate pe verticala sau orizontala (pe aceeasi linie sau aceeasi coloana).

```
  123456789
1 ±±±±±±±±±    ± - zid
2 ±       ± ±
3 ±±±I±±± ±    I - pozitia initiala
4 ±       ±F±  F- pozitia finala
5 ±±± ±   ±
6 ±±±±±±±±±
```

Solutia acestei probleme este deci un vector de pozitii in matrice, in cazul nostru solutia este:

((3,4), (4,4), (4,5), (4,6), (5,6), (5,7), (5,8), (4,8))

Fiecare pozitie din drum este rezultatul unei alegeri intre mai multe variante de inaintare. De exemplu din pozitia initiala (3,4) se alege intre variantele (2,4) si (4,4).

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 2

Metoda de rezolvare se bazeaza pe incercari. Dintr-o anumita pozitie se inainteaza atit cit este posibil, pe rind in toate directiile libere. Atunci cind o pista se "inchide" fara sa se ajunga in pozitia finala, se revine urmarind drumul parcurs pentru a incerca alte piste.

Matricea labirint va fi o matrice de caractere:

L[i,j]='±' =>zid
L[i,j]=' ' =>pozitie libera.

Pentru a putea indexa variantele de inaintare dintr-o pozitie se foloseste vectorul "***dir***" cu incrementii care trebuie aplicati indecsilor de linie si coloana pentru inaintare intr-o directie.

Directia:	**1**	**2**	**3**	**4**
Increment pt. *x*	0	0	-1	1
Increment pt. *y*	-1	1	0	0

Se poate aplica urmatoarea strategie recursiva: cautarea drumului pina la pozitia finala, pornind dintr-o pozitie data, va insemna cautarea drumului pornind pe rând din toate pozitiile invecinate din care acesta cautare nu a fost facuta anterior.

Sa se scrie o varianta de rezolvare a problemei labirintului folosind o functie C (C++) recursiva. Pentru initializarea matricii L (labirintul) veti folosi o functie *citeste_labirint* care citeste matricea labirint dintr-un fisier text, al carui nume il primeste drept parametru:

Continutul fisierului:

```
6 9
±±±±±±±±
±      ± ±
±±±I±±± ±
±      ±F±
±±± ±   ±
±±±±±±±±
```

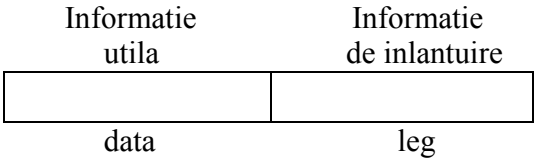
Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 2

2. Liste liniare simplu inlantuite

2.1. Introducere

O ***lista*** este o colectie de elemente intre care este specificata cel putin o relatie de ordine. O ***lista liniara simplu inlantuita*** este caracterizata prin faptul ca relatia de ordine definita pe multimea elementelor este unica si totala. Ordinea elementelor pentru o astfel de lista este specificata explicit printr-un cimp de informatie care este parte componenta a fiecarui element si indica elementul urmator, conform cu relatia de ordine definita pe multimea elementelor listei.

Deci fiecare element de lista simplu inlantuita are urmatoarea structura:



Pe baza informatiei de inlantuire (pastrata in cimpul ***leg***) trebuie sa poata fi identificat urmatorul element din lista.

Daca exista un ultim element in lista atunci lista se numeste ***liniara***. Daca nu exista un element care sa contina in cimpul informatie valoarea *null*

2.2. Lista liniara simplu inlantuita alocata static

Daca implementarea structurii de lista inlantuita se face prin *tablouri*, aceasta este o lista inlantuita alocata static sau simplu o ***lista inlantuita statica***.

Consideram urmatoarele declaratii:

```
struct Element {
           char* data;
           int leg;
           };
Element V[8];
```

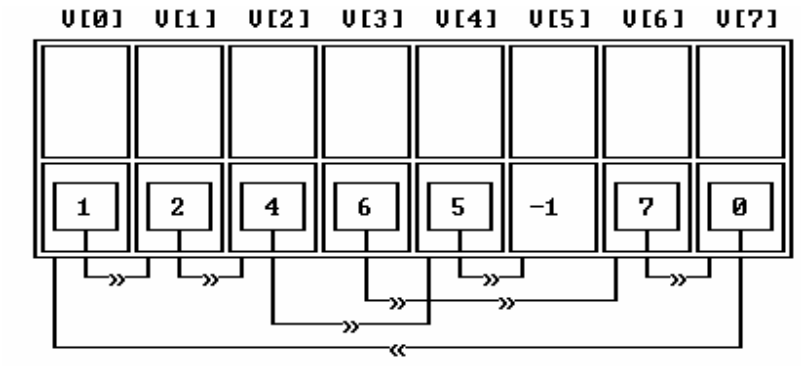
Pentru elementele vectorului ***V*** exista o ordine naturala data de aranjarea in memorie a elemetelor sale: V[0], V[1], ... V[7]. Vom reperezenta memoria ocupata de vectorul ***V*** astfel incit fiecare element sa fie reprezentat vertical, in felul urmator:

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 2

	0	1	2	3	4	5	6	7
data								
leg								

Completind cimpul ***leg*** pentru fiecare element al vectorului putem obtine o lista liniara simplu inlantuita. Valoarea cimpului ***leg*** va fi indexul in vector al urmatorului element din lista.

Vectorul V:



Pe baza inlantuirii stabilita de valorile din figura de mai sus se obtine ordinea:

V[3], V[6], V[7], V[0], V[1], V[2], V[4], V[5].

Obs. Ultimul element din lista are in cimpul ***leg*** valoarea -1. Este necesar sa cunoastem care este primul element din inlantuire, pentru aceasta retinem intr-o variabila:

```
int cap;
```

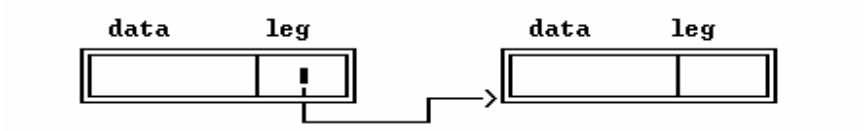
indexul primului element.

```
cap=3.
```

Parcurgerea in ordine a elemntelor listei se face in felul urmator:

```
int crt;
.................
crt = cap;
while (crt!=-1) {
                 ┌──────────────────┐
                 │ Prelucreaza V[crt] │
                 └──────────────────┘
                 crt = V[crt].leg;
                 }
```

Indiferent de modul cum se materializeaza informatiile de legatura pentru a reprezenta o lista inlantuita vom folosi urmatoarea reprezentare:



Sageata care porneste din cimpul ***leg*** arata faptul ca valoarea memorata aici indica elementul la care duce sageata.

2.3. Lista liniara simplu inlantuita alocata dinamic

Daca implementarea structurii de lista inlantuita se face prin tehnici de alocare dinamica se obtine o lista inlantuita alocata dinamic sau simplu o lista inlantuita dinamica.

Pentru rolul pe care il joaca informatiile de legatura in structurile inlantuite, cel mai potrivit este tipul *pointer*. Tipul cimpului ***leg*** va fi "***pointer la element de lista***".
Iata cum arata declaratiile tipului "element de lista liniara simplu inlantuita" in C++:

```
struct Element {
       TipOarecare data;          // informatia utila
       Element* leg;              // legatura
       };
```

In C va trebui sa scriem:

```
typedef struct _Element {
       TipOarecare data;
       struct _Element* leg;
     } Element;
```

Avind declaratiile de mai sus (una din forme), si

```
Element* p;                          // un pointer la Element
```

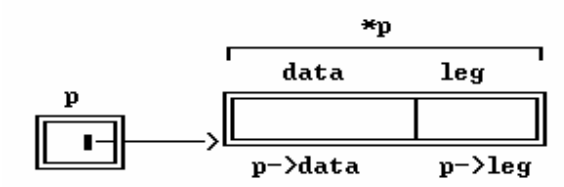
in urma unei operatii:

```
p = (Element*) malloc( sizeof(Element) );  // in C
```

sau, simplu

```
p = new Element;                          // in C++
```

p a fost initializat cu adresa unei variabile de tip ***Element*** alocata in zona de alocare dinamica:



Atunci, aceasta din urma va fi identificata prin expresia ***p** iar cimpurile sale prin expresiile **p->data** si respectiv **p->leg** .

Constanta **0** (**NULL**) pentru un pointer inseamna o adresa imposibila. Aceasta valoare va fi folosita pentru a sfirsi inlantuirea (ultimul element din lista va avea **p->leg = 0**).

Pentru a manevra o lista avem nevoie doar de un pointer la primul element al listei. Pentru a indica o lista vida acest pointer va primi valoarea 0.

2.4. Operatii in liste liniare simplu inlantuite

Fara a restringe generalitatea, vom detalia doar implementarea prin pointeri.

Consideram declaratiile de tipuri de mai sus si variabilele:

```
Element* cap;    // pointer la primul element al unei liste
Element* p;
Element* q;
```

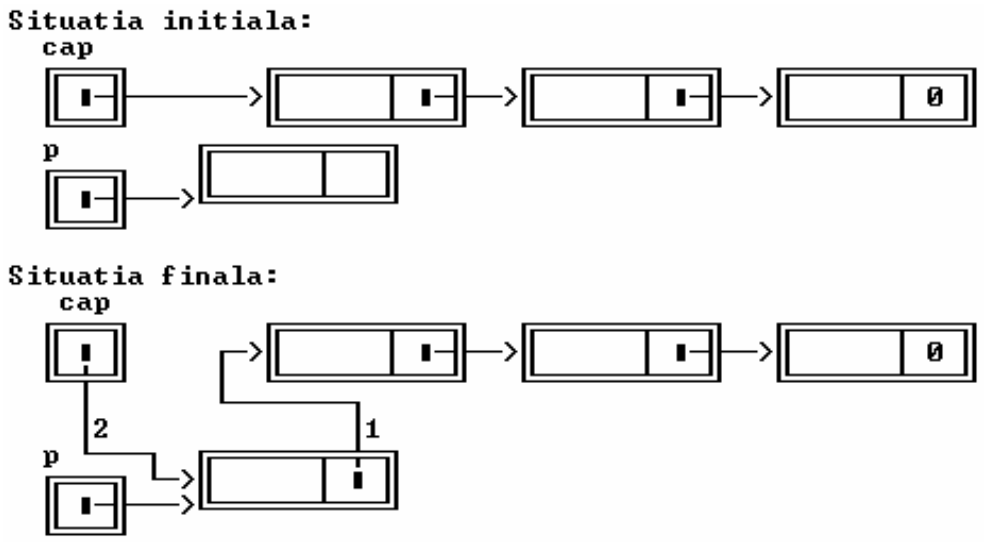
Operatiile primitive pentru acces la o lista inlantuita sint:

2.4.1. Inserarea unui element in lista

Consideram: ***cap*** - contine adresa primului element din lista;
p - contine adresa unui element izolat care dorim sa fie inserat in lista.

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 2

2.4.1.1. Inserarea in fata



Fiecare sageata nou creeata insemna o atribuire: se atribuie variabilei in care sageata nou creata isi are originea, valoarea luata dintr-o variabila in care se afla originea unei sageti cu aceeasi destinatie.

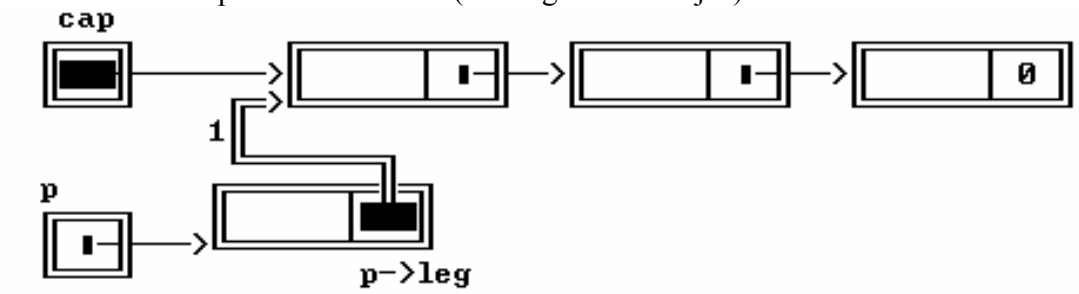
In cazul nostru avem atribuirile (fiecare atribuire corespunde sagetii cu acelasi numar din figura):

```
(1)     p->leg = cap;
(2)     cap = p;
```

Sa detaliem: Prima atribuire

```
p->leg = cap;
```

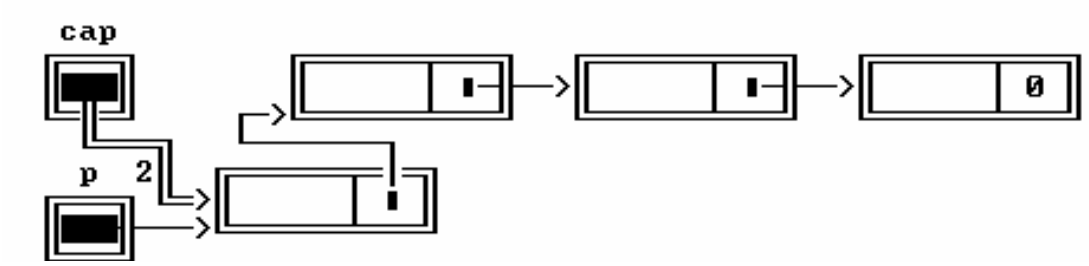
leaga elementul de inserat de restul listei. In urma acestei atribuiri, **cap** si **p->leg** vor indica ambii inceputul listei initiale (vezi figura de mai jos).



A doua atribuire:

```
cap = p;
```

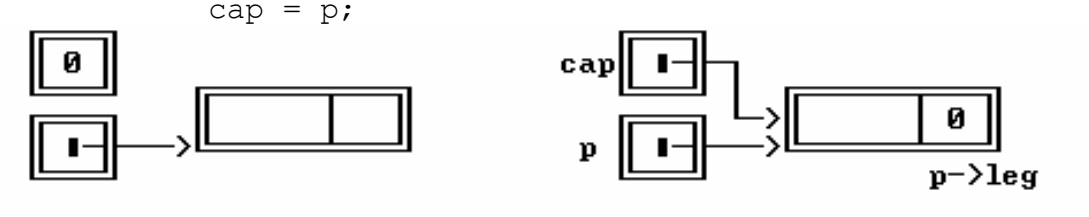
pune in pointerul cap adresa elementului inserat in fata listei.



Observatie:

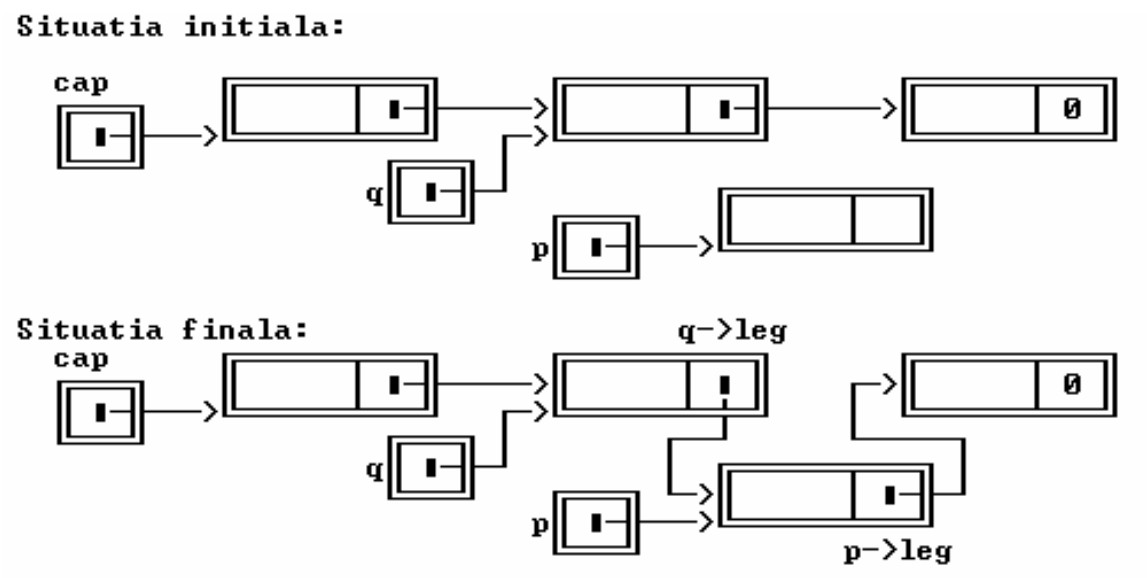
Daca pointerul **cap** este initial nul, (ceea ce inseamna inserarea intr-o lista vida) atribuirile de mai sus functioneaza corect rezultind o lista cu un singur element.

```
p->leg = cap;     // de fapt    p->leg = 0;
cap = p;
```



2.4.1.2. Inserarea in interior sau la sfirsit

Varibila **q** va indica elementul dupa care se face inserarea.




```
(1)     p->leg = q->leg;
(2)     q->leg = p;
```

Observatii:

Atunci cind **q** indica ultimul element dintr-o lista, atribuirile de mai sus functioneaza corect si adauga elementul indicat de **p** la sfirsitul listei.

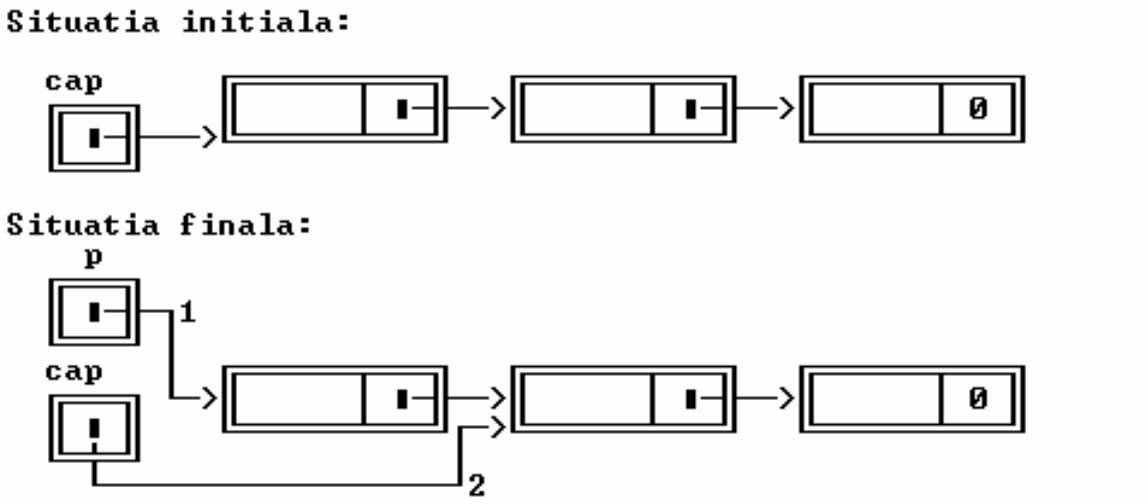
Nu se poate face inserarea in fata unui element dat (prin **q**) fara a parcurge lista de la capat.

2.4.2. Stergerea unui element din lista

Consideram: **cap** - contine adresa primului element din lista.

2.4.2.1. Stergerea la inceputul listei

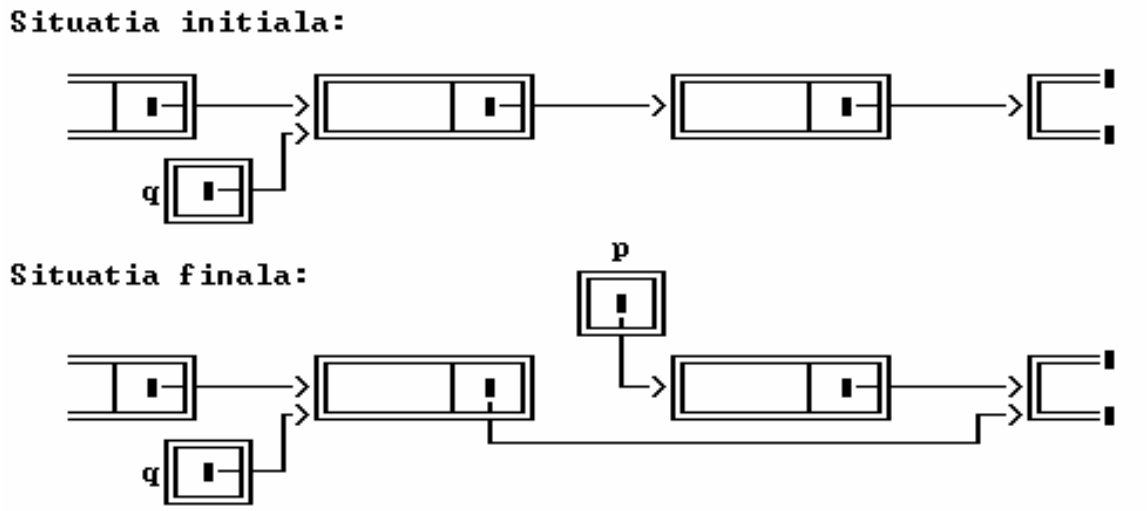
Prin operatia de stergere se intelege scoaterea unui element din inlantuire. Elementul care a fost izolat de lista trebuie sa fie procesat in continuare, cel putin pentru a fi eliberata zona de memorie pe care o ocupa, de aceea adresa lui trebuie salvata (sa zicem in variabila pointer **p**).




```
(1)     p = cap;
(2)     cap = cap->leg;
        delete p;       // Elibereaza zona de memorie
```

2.4.2.2. Stergerea in interior sau la sfirsit

Varibila **q** va indica elementul din fata celui care va fi sters.




```
(1)     p = q->leg;
(2)     q->leg = p->leg;  // sau q->leg = q->leg->leg;
        delete p;
```

Observatii:

Atunci cind **q** indica penultimul element dintr-o lista, atribuirile de mai sus functioneaza corect si sterg ultimul element din lista.

Nu se poate face stergerea elementului indicat de **q** fara parcurgerea listei de la capat.

2.4.3. Parcurgerea listei

Consideram: **cap** - contine adresa primului element din lista.

O parcurgere inseamna prelucrarea pe rind a tuturor elementelor listei, in ordinea in care acestea apar in lista. Vom avea o variabila pointer **p** care va indica pe rind fiecare element al listei:

```
p = cap;
while (p!=0){
            Prelucreaza p->data
            p = p->leg;
            }
```

```
for(p=cap; p!=0; p=p->leg){
        Prelucreaza p->data
        }
```

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 2

Un caz special apare atunci cind dorim sa facem o parcurgere care sa se opreasca in fata unui element care sa indeplineasca o conditie (ca in cazul cind inseram un element intr-o pozitie data printr-o conditie, sau stergem un elemen care indeplineste o conditie).

Presupunem ca lista are cel putin un element.

```
p = cap;
while (p->leg!=0 && !conditie(p->leg))
                    p = p->leg;
```

Bucla ***while*** se poate opri pe condifia "***p->leg==0***", ceea ce insemna ca nici un element din lista nu indeplineste conditia iar poinertul ***p*** indica ultimul element din lista, sau pe conditia "***conditie(p->leg)***", ceea ce insemna ca pointerul ***p*** va contine adresa elementului din fata primului element care indeplineste conditia.

APLICAȚII

1. Se citeste de la intrare un sir de numere intregi.

a) Sa se plaseze numerele citite intr-o lista inlantuita, prin inserari repetate in fata listei.
b) Sa se afiseze lista creata.
c) Se citeste un numar si sa se determine daca acesta se afla printre elementele listei create.
d) Sa se insereze un numar citit de la intrare intr-o pozitie citita de la intrare.
e) Sa se stearga un element din lista dintr-o pozitie citita de la intrare.
f) Sa se afiseze elementul situat pe pozitia k numarata de la sfirsitul la inceputul listei, fara a parcurge lista mai mult de o data.

2. Sa se construiasca un modul (fisierle .H si .C (.CPP)) care sa contina tipurile de date si functiile care implementeaza urmatoarele operatii:

- parcurge o listă simplu inlantuita in ambele sensuri (dus-intors) utilizind O(1) memorie suplimentara.
- testeaza daca o lista simplu inlantuita are bucle.
- determina mijlocul unei liste simplu inlantuite.

TEMA

1. Sa se construiasca modul (fisierle .H si .C (.CPP)) care sa contina tipurile de date si operatiile care implementeaza sub forma unei liste simplu inlantuite o agenda de numere de telefon. Elementele listei vor contine ca informatie utila doua cimpuri:
- nume - numele persoanei;
- tel - numarul de telefon;

Elementele listei vor fi pastrate in ordine alfabetica dupa numele persoanei.

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 2

Sa se definiesca procedurile care:
- insereaza un element in lista;
- sterge din lista o persoana data;
- cauta in lista numarul de telefon al unei persoane date;
- afiseaza lista in intregime.

2. Fie X=(x[1],x[2],...,x[n]) si Y=(y[1],y[2],...,y[m]) doua liste liniare simplu inlantuite. Scrieti un program C (C++) care:
- sa uneasca cele doua liste in una singura:
  Z=(x[1],x[2],..,x[n],y[1],y[2],...,y[m])
- sa interclasese cele doua liste asfel:
  Z=(x[1],y[1],x[2],y[2],...,x[m],y[m],x[m+1],...,x[n]) daca m<=n sau
  Z=(x[1],y[1],x[2],y[2],...,x[n],y[n],y[n+1],...,y[m]) daca n<=m